2025/2026

UniRide

Technical Documentation

ISPW Course Project

Software Engineering

Alessio Carta

INDEX

1. SOFTWARE REQUIREMENT SPECIFICATION 3
    1.1. Introduction 3
    1.2. User Stories 4
    1.3. Functional Requirements 5
    1.4. Use Cases 6
2. STORYBOARDS 7
3. DESIGN 9
    3.1. Class Diagram — VOPC (Analysis) 9
    3.2. Class Diagram — Design Level 15
    3.3. Design Patterns 16
    3.4. Activity Diagram 17
    3.5. Sequence Diagram 19
    3.6. State Diagram 19
4. TESTING 20
DELIVERABLE LINKS 21

Page numbers refer to the PDF export of this document; they may vary slightly when opening the file in Word, depending on the fonts available on the system.

1. SOFTWARE REQUIREMENT SPECIFICATION

1.1. Introduction

1.1.1. Aim of the Document

This document aims to objectively present the fundamental and specific requirements of the UniRide system. Through textual descriptions, user stories, use case diagrams and design artifacts, all relevant internal and external aspects of the system are covered, serving as an authoritative reference for development, testing and evaluation.

1.1.2. Overview of the Defined System

The system described here is a desktop application that helps university students organize shared car rides to and from campus. The primary goal is to reduce travel costs and CO_2 emissions, and to foster socialization among students of the same university.

The system supports two roles:

- Driver (Student) — publishes a new ride specifying departure, destination, date, available seats and an estimated base cost. Incoming booking requests are explicitly confirmed or rejected by the driver; the ride can also be marked as completed or cancelled at any time.
- Passenger (Student) — browses available rides, filtering by departure and destination, and requests a seat. The seat is reserved immediately to prevent overbooking, but the request remains pending until the driver confirms or rejects it; if the driver does not answer within 24 hours, the request expires automatically and the seat is released. The passenger can cancel a request or an already-confirmed booking at any time, automatically freeing the seat.

The system manages the complete lifecycle of a Ride (AVAILABLE, FULL, COMPLETED, CANCELLED, via the State Pattern) and of a Booking (REQUESTED, CONFIRMED, REJECTED, CANCELLED), notifying the involved passengers at every relevant transition via the Observer Pattern.

1.1.3. Hardware and Software Requirements

The basic hardware requirement is a computer capable of running a Java Virtual Machine, plus a small amount of disk space for CSV files when the file-system persistence mode is active.

The software requirements are as follows:

Requirement	**Detail**
JDK	Java 25 or later (OpenJDK or Oracle JDK)
Build Tool	Apache Maven 3.8 or later
GUI Toolkit	JavaFX 21.0.5 (automatically resolved by Maven)
Operating System	macOS, Windows 10+, or Linux (Ubuntu 20+)
RAM	Minimum 512 MB free
Disk Space	About 50 MB (JAR plus any CSV files)
Persistence	In-Memory (default), CSV file system or H2 relational database, selectable at startup with -Duniride.persistence (Config.PERSISTENCE_TYPE)

1.1.4. Related Systems, Pros and Cons

BLABLACAR

BlaBlaCar is the leading European carpooling marketplace, connecting millions of drivers and passengers for intercity trips. Pros: a very large user base, an integrated payment system, and verified driver profiles. Cons: it is not designed for students or university routes, it requires a financial transaction for every booking, and it has no concept of a university campus as a reference point.

GOMORE

GoMore is a Scandinavian ride-sharing platform that also supports peer-to-peer car rental. In its favor, it offers flexible ride listings and a clean, modern mobile interface. However, it lacks features tied to an academic community (no university filter, no student verification), imposes a subscription model on drivers, and offers no offline or in-memory demo mode, making it a poor reference for a BCE-style Java desktop application.

1.2. User Stories

1.2.1. US-1

US-1

As a Student, I want to publish a ride offer specifying departure, destination, date, available seats and total cost, so that other students can find it and request a seat in my car.

1.2.2. US-2

US-2

As a Student, I want to search for available rides by filtering on departure and destination, so I can quickly find a suitable ride without scrolling through irrelevant offers.

1.2.3. US-3

US-3

As a Student, I want to request a seat on a ride and be notified when the driver confirms or rejects it, so I always know the real status of my booking.

1.2.4. US-4

US-4

As a Student driver, I want to be able to confirm or reject incoming booking requests, mark a ride as completed, or cancel it even with passengers already on board, so I have full control over my ride.

1.3. Functional Requirements

1.3.1. FR-1

FR-1 — Authentication

The system shall register a student with a username, password and personal details.

1.3.2. FR-2

FR-2 — Ride Management

The system shall publish a ride on behalf of an authenticated driver.

1.3.3. FR-3

FR-3 — Booking

The system shall search available rides upon an authenticated passenger's request.

1.3.4. FR-4

FR-4 — Request Management

The system shall confirm a booking request on behalf of the driver.

Glossary:

Ride = A trip offered by a student driver, with departure, destination, date, time, available seats and base cost.

Booking = A passenger's request to occupy a seat on a Ride: the seat is reserved immediately, but remains pending the driver's confirmation.

Nearest University = The university campus, among those known to the system, geographically closest to the student's declared location; suggested as the default destination in Offer/Search Ride.

Ride State = The lifecycle phase of a Ride (AVAILABLE, FULL, COMPLETED, CANCELLED), governed by the State Pattern.

Booking State = The lifecycle phase of a Booking (REQUESTED, CONFIRMED, REJECTED, CANCELLED, EXPIRED).

1.4. Use Cases

1.4.1. Overview Diagram

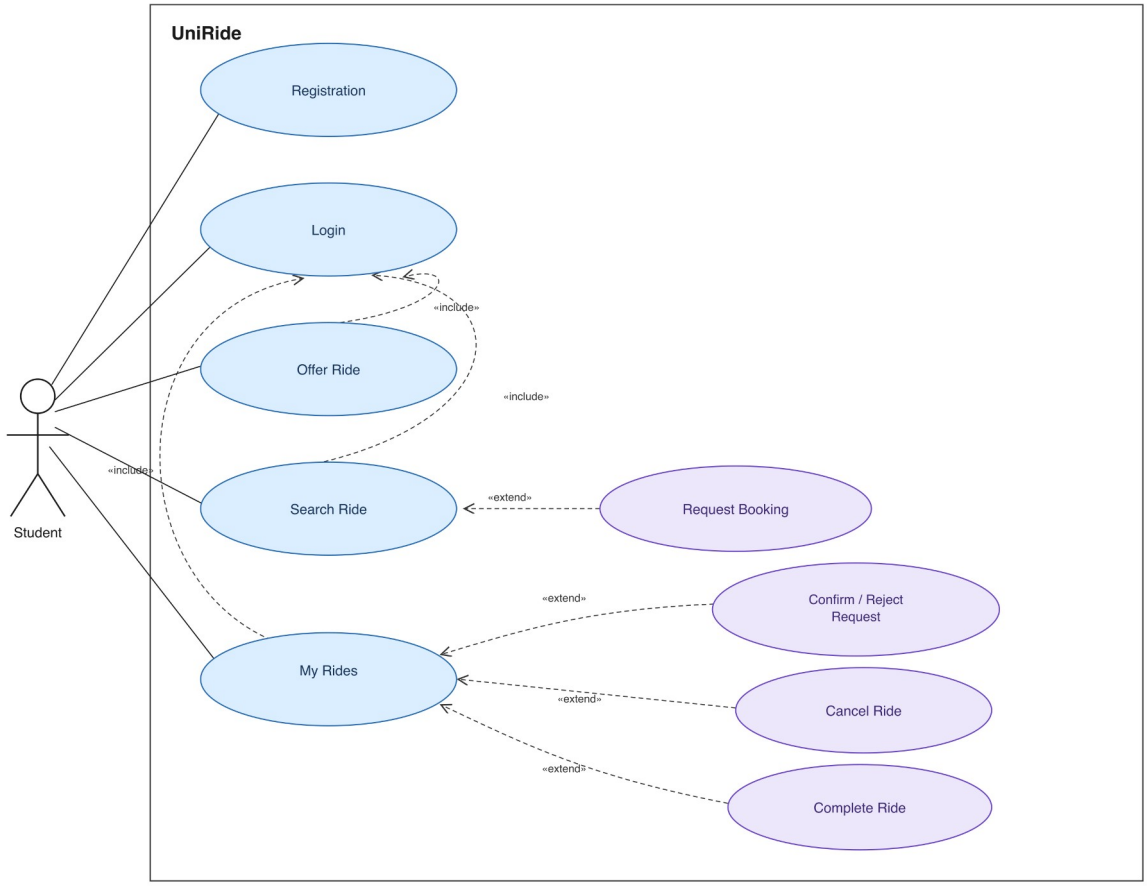


Figure 1 — Use case diagram, Student actor

1.4.2. Internal Steps

Name: Offer a new ride

- 1) The driver opens the 'Offri un Passaggio' screen from the Dashboard.
- 2) The driver selects departure and destination from a location dropdown (editable for free-text search), a date from the calendar, a departure time (HH:mm format), the available seats with a numeric counter, and the base cost.
- 3) The system checks the form of every field as soon as it is entered (mandatory fields, real date in the dd/MM/yyyy format, time in the HH:mm format, numeric seats and cost), then the business rules: departure and destination must not coincide, the seats must be more than zero, the cost must not be negative and the date must not be in the past.
- 4) The system creates the Ride entity, generating an internal UUID and setting the initial state to AVAILABLE, then persists it through the RideDAO obtained from the DAOFactory.
- 5) The system shows a confirmation message and clears the form.

Extensions:

- 3a. A required field is empty, or departure and destination coincide: the system shows an inline error and stays on the same screen without persisting anything.

- 3b. The number of seats is not greater than zero: the system rejects the input with the message 'Il numero di posti deve essere maggiore di zero.'
- 3c. The entered cost is negative: the system rejects the input with the message 'Il costo base non può essere negativo.'
- 3d. The departure time is missing or does not match the HH:mm format: the system rejects the input with the message 'Inserisci un orario di partenza valido (formato HH:mm, es. 08:30).'
- 3e. The date is missing, is not a real calendar date, or is already in the past: the system rejects the input with the message 'La data del passaggio è obbligatoria.', 'Data non valida: usa il formato gg/mm/aaaa (es. 15/03/2027).' or 'Non si può offrire un passaggio in una data già passata.' respectively.

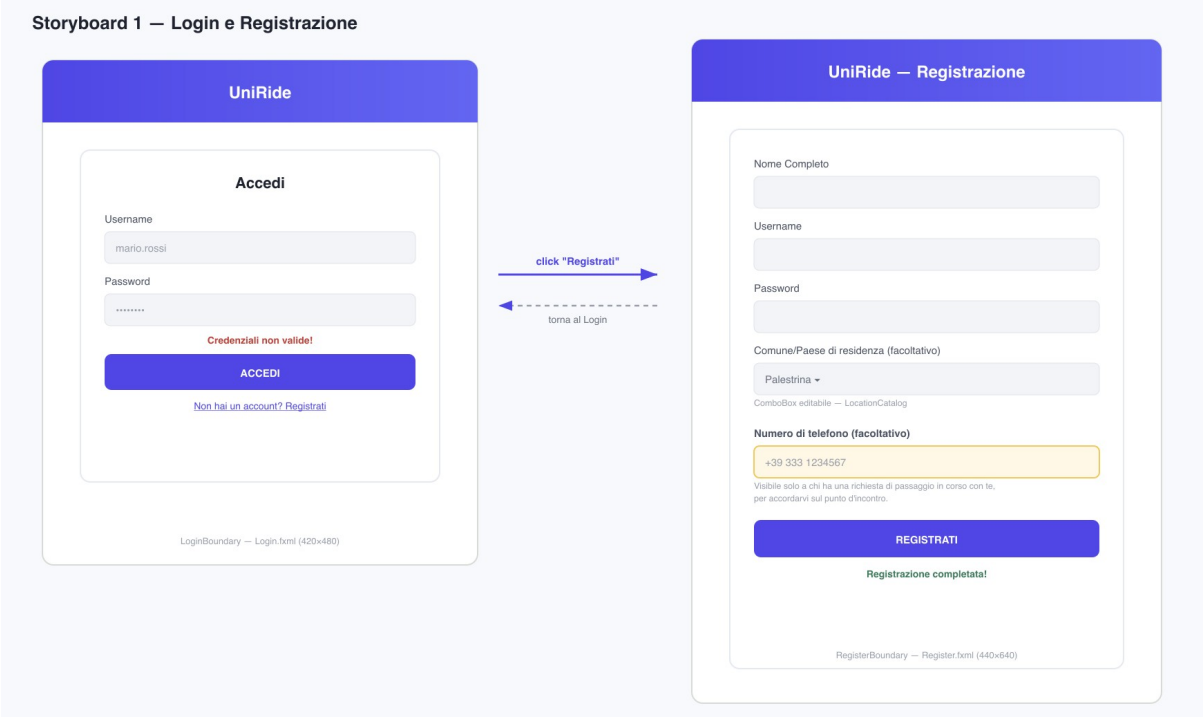
Name: Request a booking

- 1) The passenger searches for available rides on a route; the system shows the matching results and the passenger selects one.
- 2) The system checks that the requester is not the ride's driver and does not already have an active request on the same ride.
- 3) The system immediately reserves the seat on the Ride (State Pattern) to prevent overbooking while the request is pending.
- 4) The system creates a Booking in the REQUESTED state and persists it through the BookingDAO.
- 5) The system shows the passenger the message 'Richiesta inviata! In attesa di conferma del guidatore.'; the request stays pending until the driver answers in the 'Le Mie Corse' use case or, after 24 hours without an answer, expires and the seat is released.

2. STORYBOARDS

The first Storyboard is the authentication screen, where the student logs in with their credentials or navigates to the registration form to create a new account.

Storyboard 1 — Login and Registration

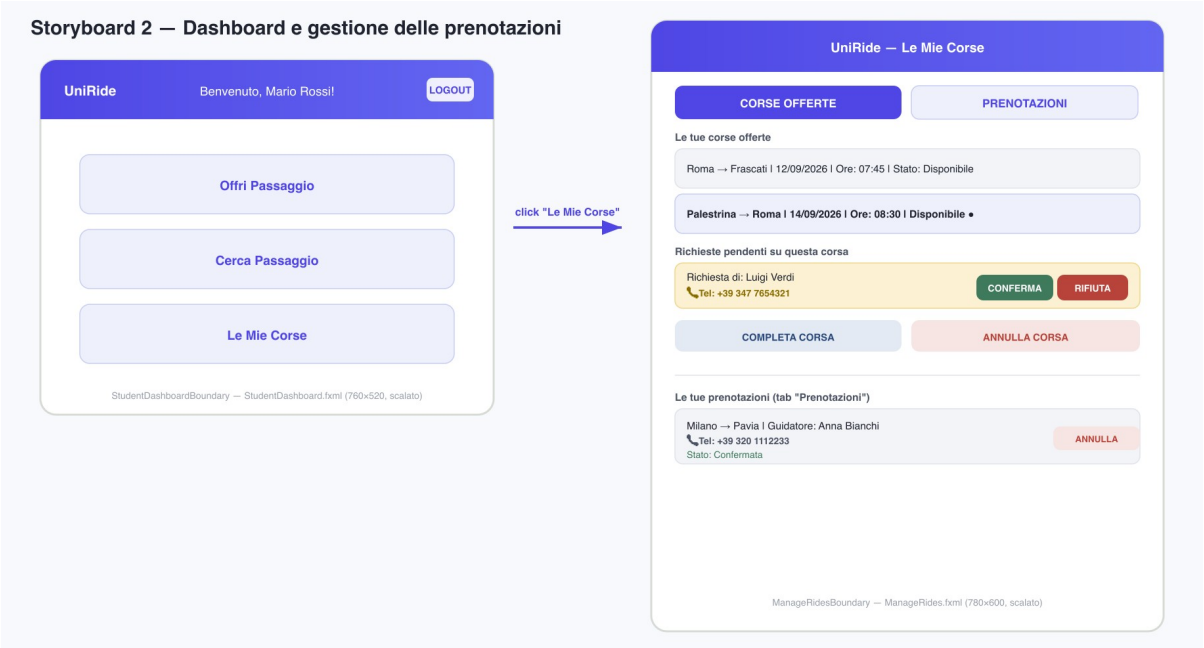


Mockup 1 — Login and Registration Storyboard (LoginBoundary / RegisterBoundary)

The login screen uses the modern design system introduced in the project: a light background with a rounded, shadowed central card, and an indigo/violet palette (#4f46e5 → #6366f1) consistent across the whole application. Two fields collect username and password. An error label, initially empty, appears in red below the fields when credentials are rejected by the LoginController. A secondary button leads to the registration form, where first and last name, username, password and — optionally — the home town/municipality (selectable from a location dropdown) are collected and validated before a Student entity is persisted.

The second Storyboard is the main Dashboard, from which all the system's features can be reached.

Storyboard 2 — Dashboard and booking management

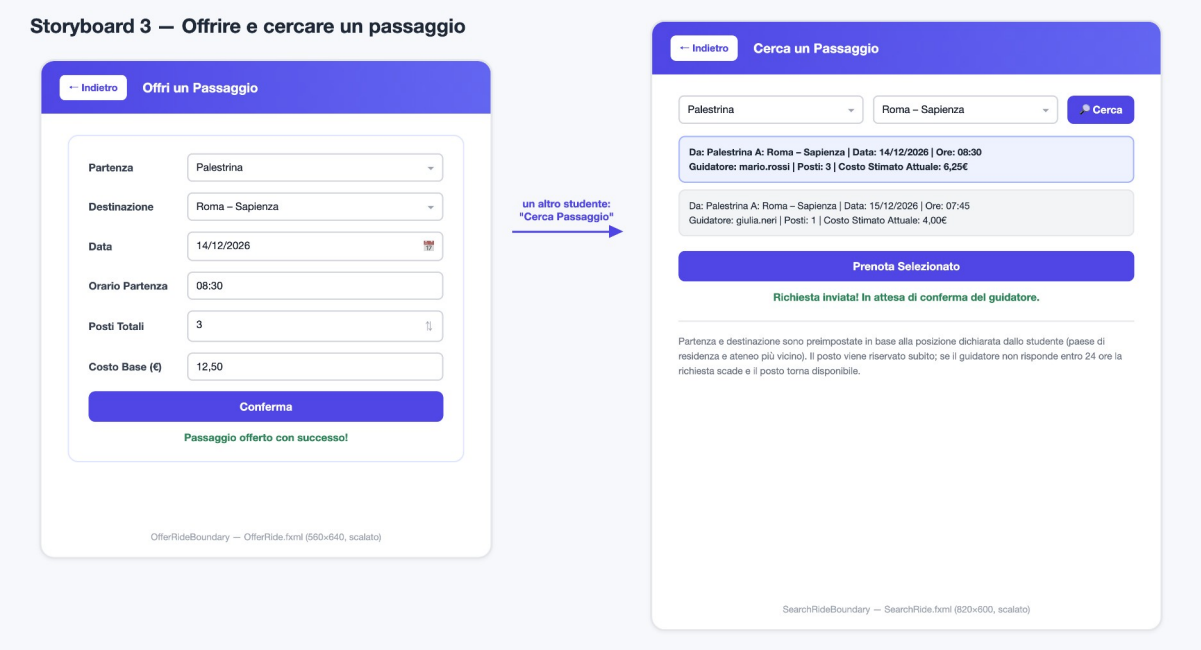


Mockup 2 — Dashboard and My Rides Storyboard (StudentDashboardBoundary / ManageRidesBoundary)

The Dashboard is built on a JavaFX BorderPane. The top bar, with an indigo/violet gradient background, shows the application logo, a welcome label personalized with the logged-in student's name, and a Logout button aligned to the right. The center of the layout presents three large navigation buttons: Offri Passaggio, Cerca Passaggio and Le Mie Corse, each of which loads its own FXML view, replacing the current Scene on the Stage. The 'Le Mie Corse' screen is organized into two tabs: the first lists the rides offered as a driver, with pending booking requests and the Confirm/Reject, Complete Ride and Cancel Ride buttons; the second lists active bookings as a passenger, with their status (Pending/Confirmed) and a button to cancel them.

The third Storyboard covers the two core features of the application: offering a ride as a driver and searching for one as a passenger.

Storyboard 3 — Offering and searching for a ride



Mockup 3 — Offer Ride and Search Ride Storyboard (OfferRideBoundary / SearchRideBoundary)

In the Offri Passaggio screen the driver chooses departure and destination from editable dropdowns, already sorted by distance from the declared home location, and fills in date, departure time, seats and base cost; every field is checked by the RideBean setters before the request reaches OfferRideController. In the Cerca Passaggio screen departure and destination are preset to the student's town and the nearest university; each result shows date, time, driver, free seats and the cost estimated by the Strategy Pattern, and 'Prenota Selezionato' sends the booking request. The seat is reserved immediately and is released automatically if the driver does not answer within 24 hours.

3. DESIGN

This section presents the system's main internal characteristics, with diagrams to illustrate them.

3.1. Class Diagram — VOPC (Analysis)

A VOPC (View Of Participating Classes) shows the classes that take part in a single use case. The analysis-level class diagram is therefore presented first for the Request Booking use case, the core of

the application (Figure 2); the combined diagram of the whole system follows (Figure 3), with classes grouped into the colored bands of the Boundary-Control-Entity pattern plus the Bean layer for the DTOs that cross architectural boundaries. The DAO families, too numerous to remain legible in the same diagram, are illustrated separately right after; a global overview of how all the layers interact closes the section.

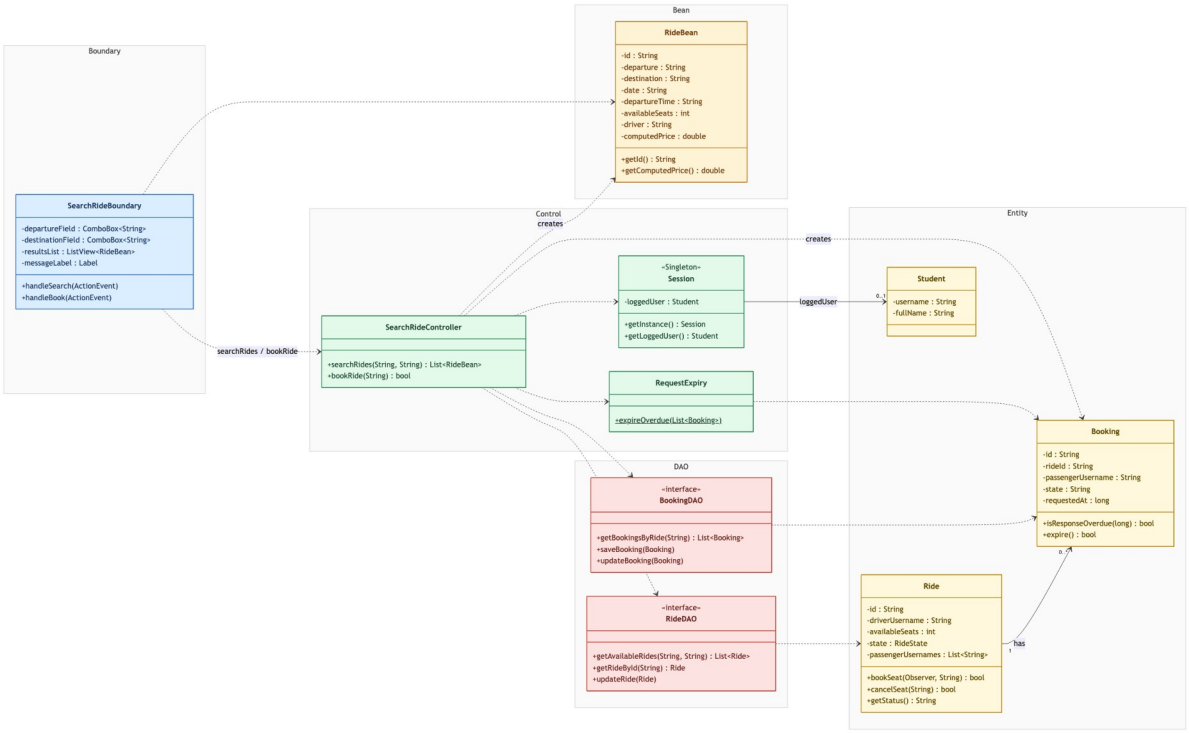


Figure 2 — VOPC of the Request Booking use case

SearchRideBoundary collects the route and the selected ride and invokes SearchRideController, which realizes the internal steps of the use case: it reads the logged-in student from the Session, lets RequestExpiry expire the requests left unanswered for 24 hours, checks that the requester is not the driver and has no active request, reserves the seat on Ride (State Pattern), creates the Booking and persists both through the RideDAO and BookingDAO interfaces. Results travel back to the Boundary only as RideBean objects, never as Entities.

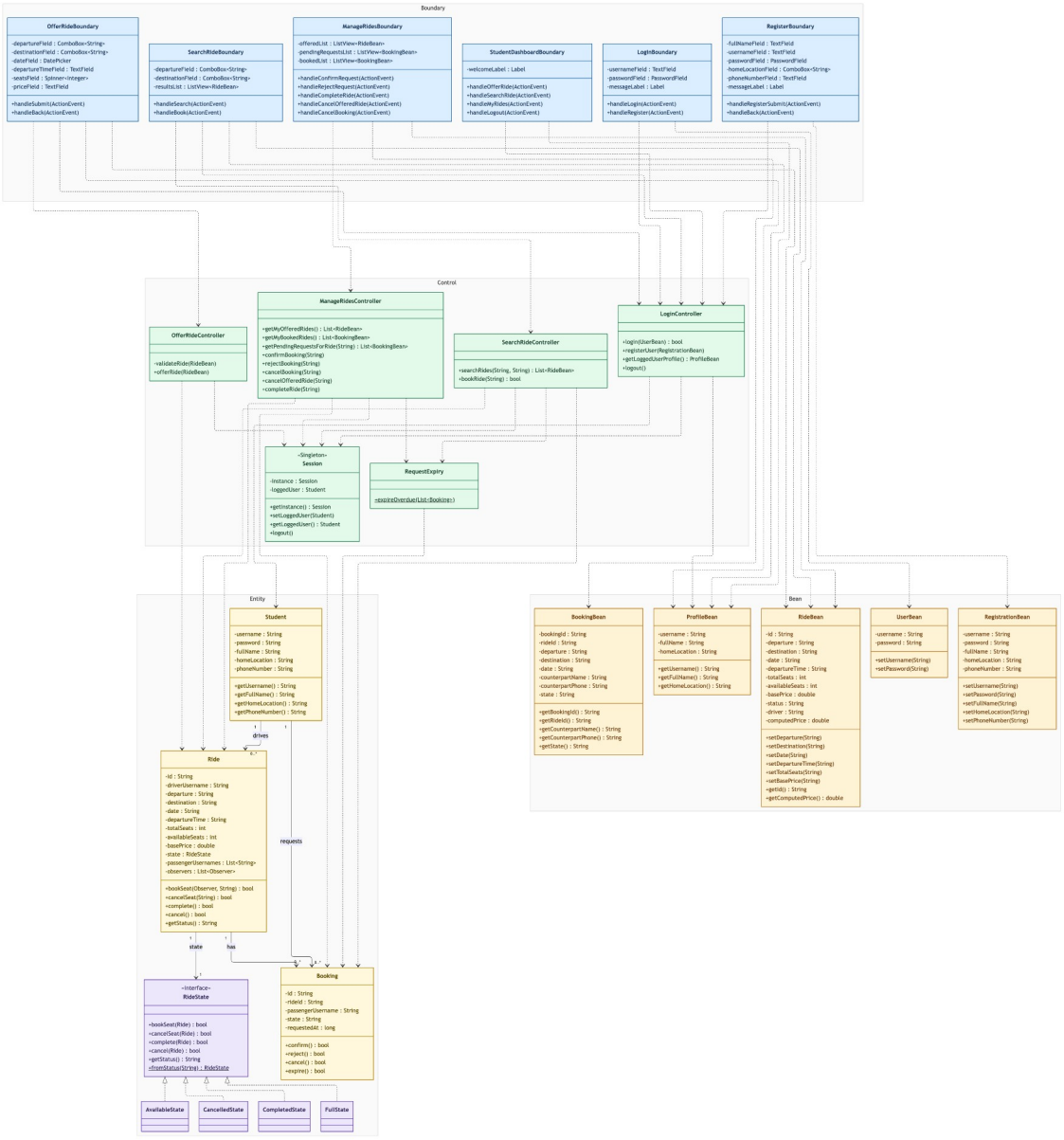


Figure 3 — Combined class diagram: Boundary, Bean, Control and Entity

The Boundary layer contains exclusively classes that interact with the user: the JavaFX controllers bound to their respective FXML views exchange only Bean objects with the Control layer, never Entities directly. Each Boundary manages the loading of its own FXML and the Scene swap on the Stage by itself: there is no centralized SceneManager class (CliMain and the various *CLI classes offer an equivalent text-based interface for headless environments, omitted here to keep the diagram readable). The Bean layer holds the DTOs that cross the Boundary ↔ Control boundary (UserBean, RegistrationBean, RideBean, BookingBean, ProfileBean): following the MVC scheme of the course, the input Beans have an empty constructor and setters that check the form of the data (mandatory fields, date and time format, numbers that can be parsed, no commas), throwing InvalidInputException, while no Entity ever leaves the domain layer toward the UI. The Control layer encapsulates the business logic: each controller receives Beans already checked syntactically, applies the business rules, orchestrates the Entities and delegates persistence to the DAO layer (illustrated separately right after); none of these classes depends on JavaFX, which lets the same controllers serve both the GUI and the CLI without modification. BCE suggests one control class per use case; UniRide groups closely related use cases in the same controller, with one public method per use case: LoginController (Registration, Login), OfferRideController (Offer Ride), SearchRideController

(Search Ride, Request Booking) and ManageRidesController (My Rides and its extensions Confirm/Reject Request, Cancel Ride, Complete Ride). The Session Singleton provides every controller with the identity of the currently authenticated student. The Boundaries never access it directly: the data they need about the logged-in student (name and home location) are exposed by LoginController.getLoggedUserProfile() as a ProfileBean, which, unlike the Student entity, carries no password. Finally, the Entity layer represents the domain classes: Ride is the richest entity, with the list of registered observers and the methods that trigger its own state transitions (bookSeat, cancelSeat, complete, cancel); Booking models the approval status of each individual booking request, including the time of the request used for the 24-hour expiry, without duplicating the seat-reservation logic already handled by Ride.

The DAO section — Abstract Factory:

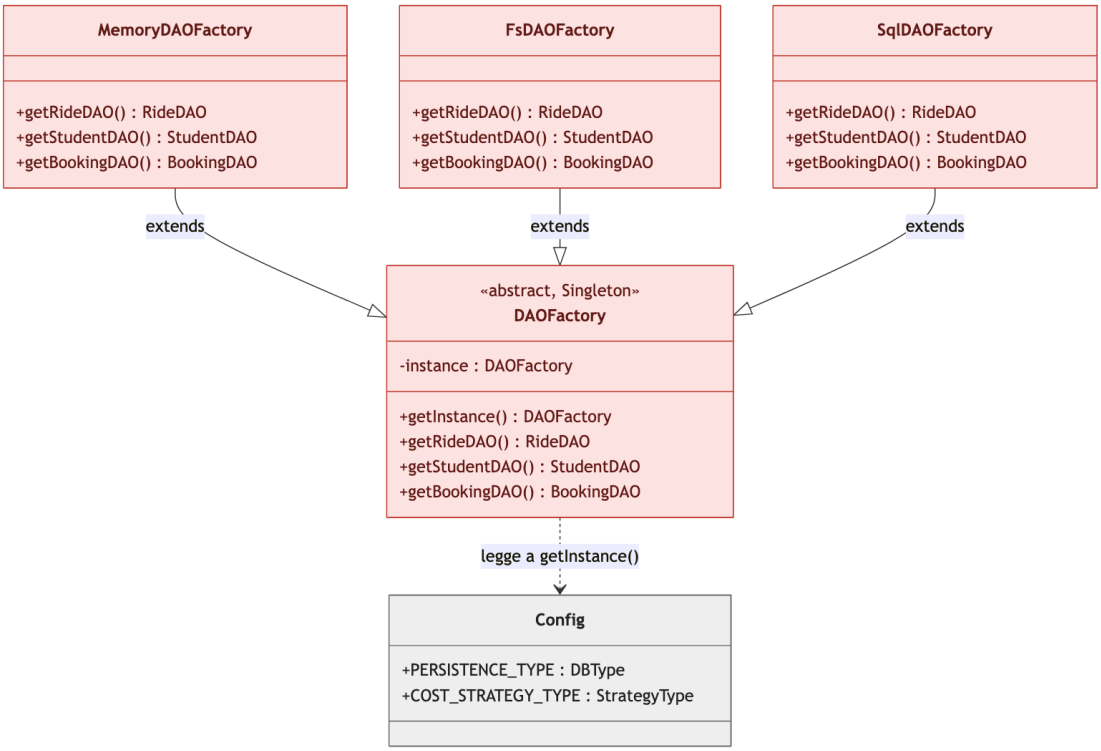


Figure 4 — DAOFactory and its Concrete Factories

DAOFactory is the single entry point into the persistence layer. It reads the PERSISTENCE_TYPE value from Config (resolved once at startup from the uniride.persistence system property, In-Memory by default) on the first call to getInstance() and caches the corresponding concrete factory, so that every controller in the system receives DAO objects belonging to the same consistent family.

The DAO section — In-Memory family:

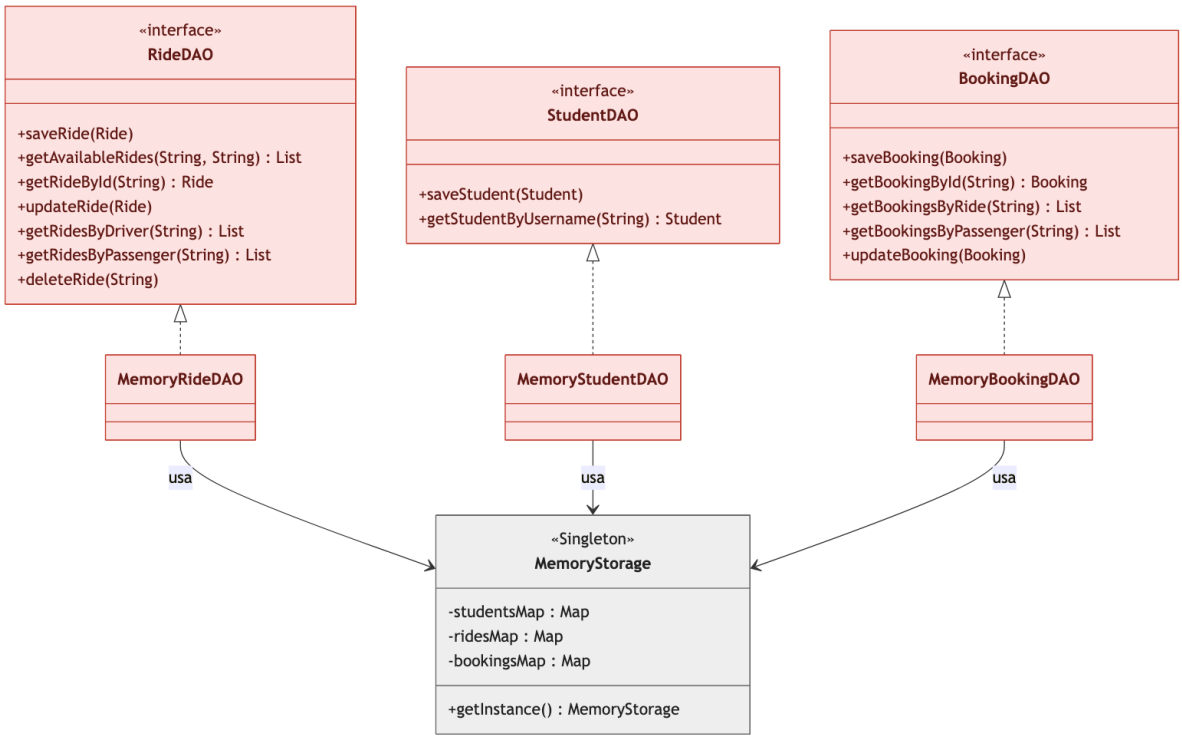


Figure 5 — In-Memory DAO

The In-Memory implementations back every interface with a static HashMap shared through the MemoryStorage Singleton, indexed by entity identifier, guaranteeing constant-time access. It is the family returned in the application's default persistence mode.

The DAO section — File System family:

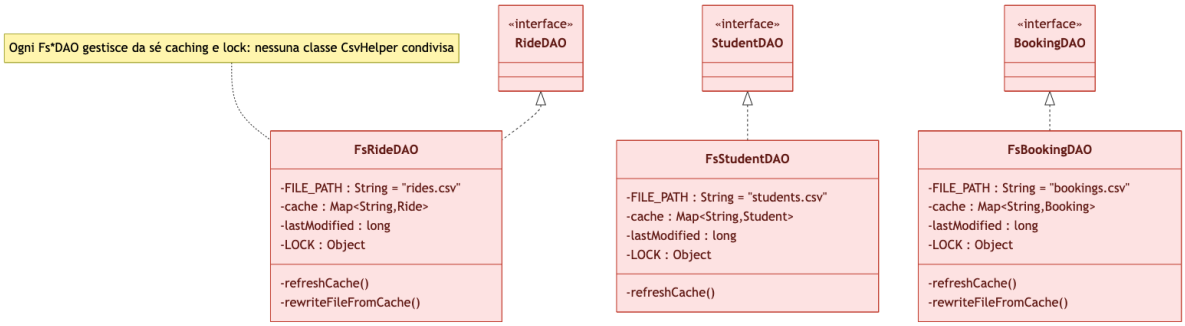


Figure 6 — File System DAO

The File System implementations each independently manage their own CSV file, with in-memory caching invalidated based on the last-modified timestamp (and discarded after a failed write, so that it is reloaded from the file, which remains the source of truth), and an explicit lock for write concurrency: there is no CsvHelper class shared among the three implementations.

The DAO section — SQL family (relational database):

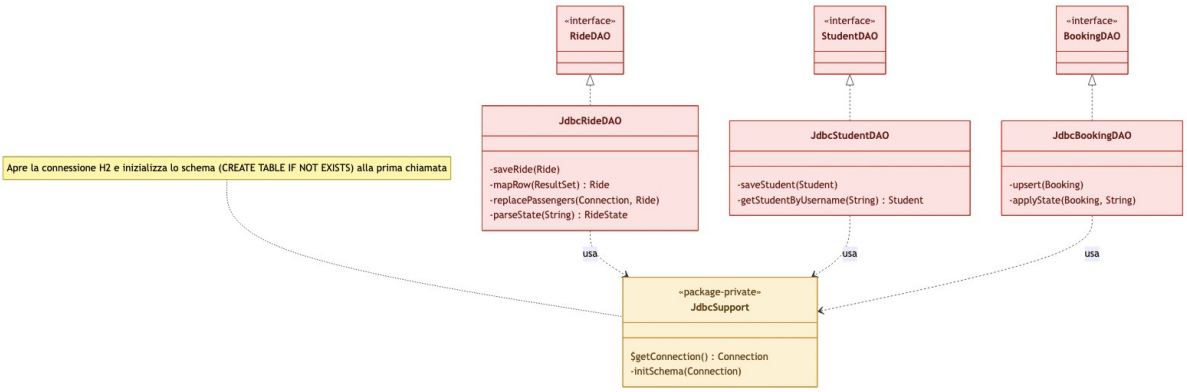


Figure 7 — SQL DAO (H2 via JDBC)

The SQL implementations rely on JdbcSupport, a package-private support class that opens the JDBC connection to the embedded H2 database and initializes its schema (CREATE TABLE IF NOT EXISTS) on first request. Each Jdbc*DAO translates its operations into prepared statements: writes use H2's own MERGE INTO … KEY (…) VALUES (…) upsert syntax, while JdbcRideDAO also manages, within a single transaction, the ride_passengers join table, needed to represent the many-to-many relationship between a ride and its passengers without flattening it into a column as in the CSV variant.

The global view of the class diagram:

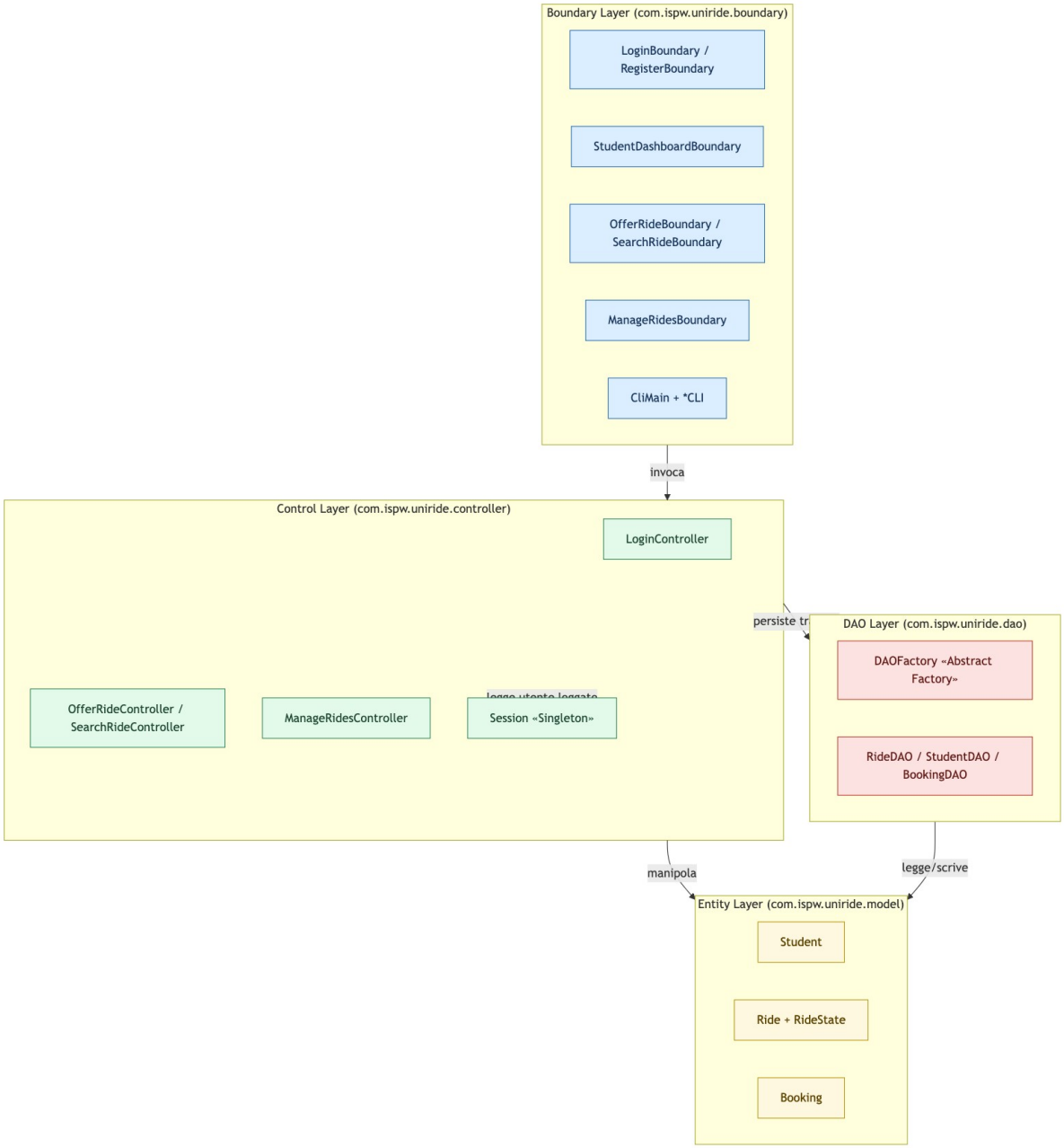


Figure 8 — Package-level architectural overview

3.2. Class Diagram — Design Level

The design-level diagram highlights the applied patterns that most raise the system's engineering level: the Observer Pattern, which decouples passenger notification from the Ride itself, and the Strategy Pattern, which makes the cost-splitting algorithm interchangeable.

The Pattern section:

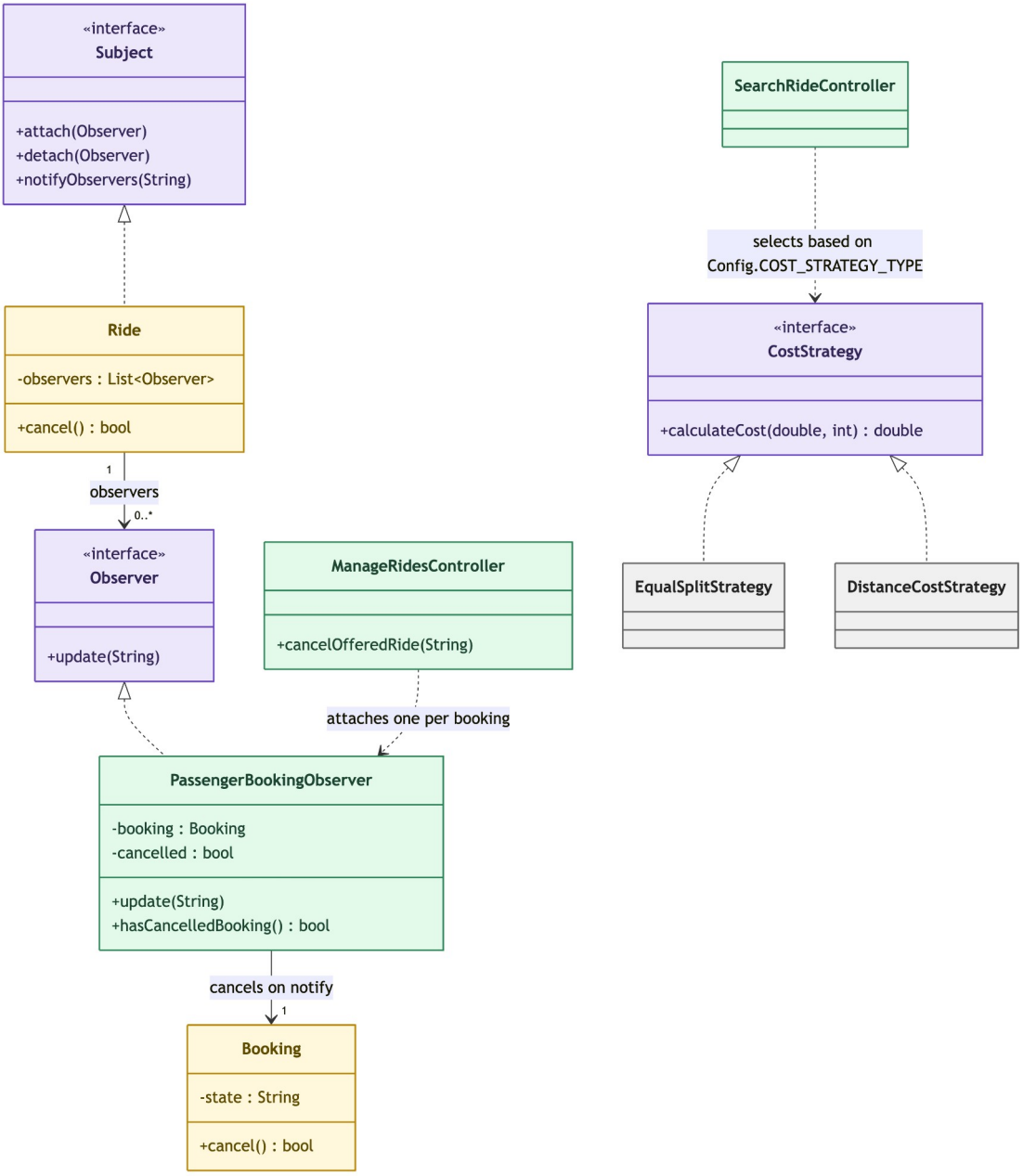


Figure 9 — Observer and Strategy applied to Ride and cost calculation

On the left, Ride plays the role of Subject and keeps a list of Observer references without knowing their concrete type (in the code, ManageRidesController registers a PassengerBookingObserver for each booking before cancelling the ride, while the observer registered at request time is a simple lambda that only logs the event). On the right, SearchRideController dynamically chooses which CostStrategy to apply based on Config.COST_STRATEGY_TYPE, allowing a new pricing model to be added without touching the existing code.

3.3. Design Patterns

Abstract Factory — DAOFactory

The system implements the Abstract Factory Pattern, whose main purpose is to provide an interface for creating families of related objects without specifying their concrete classes. DAOFactory is an abstract class that exposes three factory methods: getRideDAO(), getStudentDAO() and

getBookingDAO(). Three concrete subclasses extend it and instantiate the correct implementations based on the value of Config.PERSISTENCE_TYPE: MemoryDAOFactory (in-RAM HashMap), FsDAOFactory (CSV files) and SqlDAOFactory (relational H2 database via JDBC). Thanks to this design, switching the entire application from one persistence technology to another requires changing a single setting (the uniride.persistence system property), and no Controller class is aware of which technology is actually storing the data.

Singleton — Session, MemoryStorage and DAOFactory

The Singleton Pattern guarantees that one and only one instance of a given object type exists, so that all actions on that object are performed consistently. Session holds the currently authenticated Student and is globally reachable via Session.getInstance(). MemoryStorage centralizes the HashMaps shared by the in-memory DAO family. DAOFactory itself caches the chosen concrete factory in a static field protected by a synchronized accessor.

State — RideState

The State Pattern models the lifecycle of a ride. The RideState interface defines the four concrete states a Ride can take on (AvailableState, FullState, CompletedState, CancelledState), and every transition is triggered exclusively by a method on Ride itself (bookSeat, cancelSeat, complete, cancel), so the state field can never diverge from the actual number of available seats or from disallowed transitions (the two terminal states, COMPLETED and CANCELLED, reject any further transition). The Booking entity instead adopts a lighter approach: a textual state with transitions validated directly by its own confirm()/reject()/cancel()/expire() methods, without a dedicated State class hierarchy. The static RideState.fromStatus(String) method, shared by the File System and SQL DAOs, rebuilds the concrete state from the label stored on disk, logging any unrecognized value instead of silently ignoring it.

Strategy — CostStrategy

The Strategy Pattern encapsulates a family of interchangeable algorithms. CostStrategy declares the single method calculateCost(basePrice, numPassengers). EqualSplitStrategy divides the total cost equally between driver and passengers, while DistanceCostStrategy calculates the cost based on distance. SearchRideController dynamically chooses which strategy to instantiate based on Config.COST_STRATEGY_TYPE on every search, so the algorithm can be changed by modifying a single constant.

Observer — Ride and its subscribed passengers

The Observer Pattern defines a one-to-many dependency so that, when an object changes state, all its dependents are automatically notified. Ride keeps a list of Observer instances and invokes notifyObservers() whenever a seat is requested, freed, or the ride is cancelled. When the driver cancels a ride, ManageRidesController attaches a PassengerBookingObserver for every booking on that ride: the notification sent by Ride.cancel() moves each booking to CANCELLED, so the passenger no longer sees it as pending or confirmed, and the Controller persists the bookings only after the ride itself has been saved. The observers are recreated on every call, because the list of subscribers lives only in memory and is not persisted by the DAOs, whereas the bookings are. The Observer registered when a seat is requested remains a lambda that logs the notification via LoggerCustom.

3.4. Activity Diagram

The following activity diagram describes the Request Booking use case. After the search, the system validates the request and, when it is valid, reserves the seat and creates the booking. A fork then starts two concurrent flows: waiting for the driver's response and a 24-hour timer; after the join, the decision distinguishes the expiry of the timer from the driver's confirmation or rejection. In the code,

the expiry is evaluated by RequestExpiry whenever the bookings of a ride are read, without a background thread:

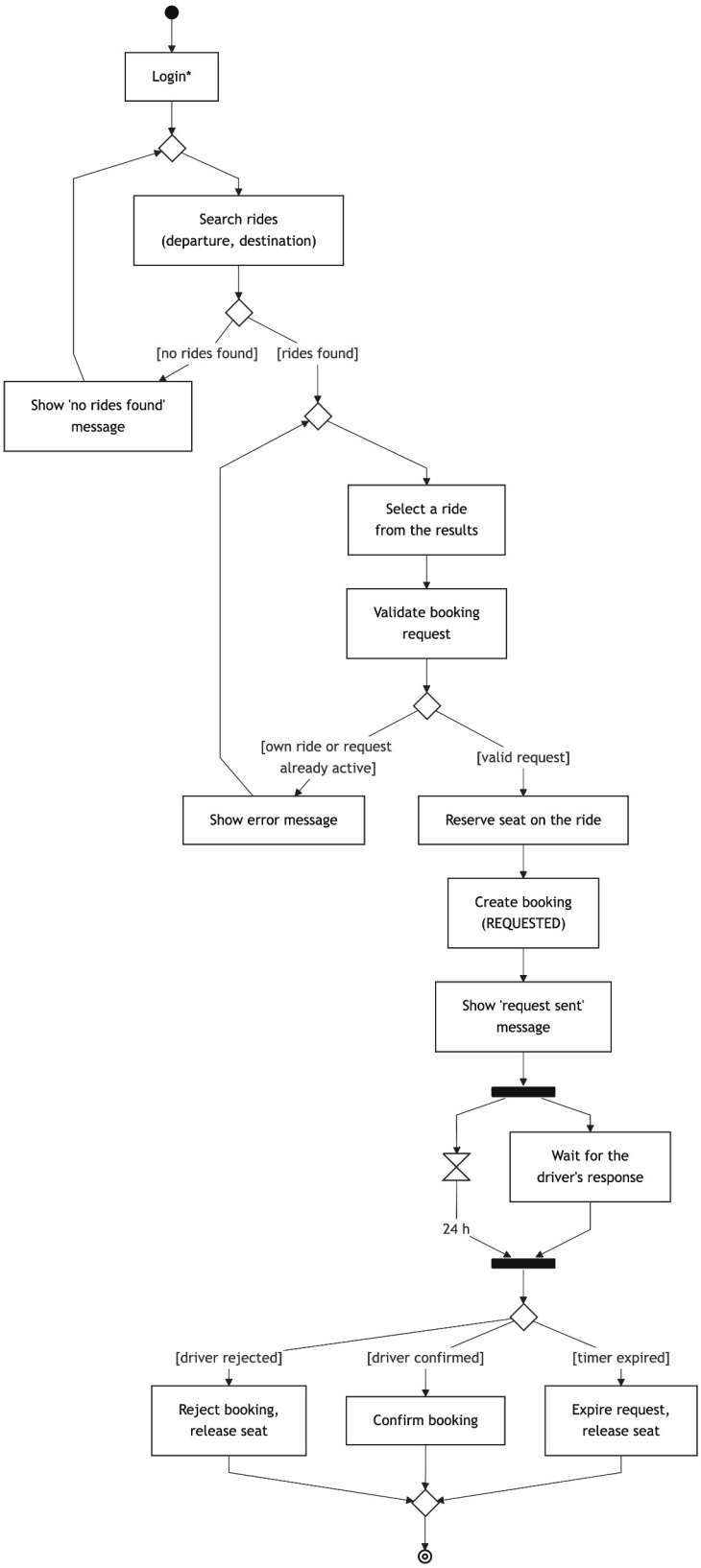


Figure 10 — Activity diagram of the Request Booking use case

3.5. Sequence Diagram

The system's main branch is described in the following sequence diagram, extended from login to the booking request. It opens with the usual chain of synchronous «create» calls from the actor to the controllers; the student then logs in (the UserBean is filled by the Boundary, LoginController verifies the PBKDF2 hash via StudentDAO and opens the session), navigates to 'Cerca Passaggio' and obtains the available results from RideDAO. When a ride is selected, SearchRideController reads the session, expires the requests older than 24 hours, validates the request, reserves the seat on Ride, creates a Booking in the REQUESTED state and persists it. Synchronous calls are drawn with a solid line and a filled arrowhead, returns with a dashed line, and the Observer notification sent by Ride with a solid line and an open arrowhead (asynchronous call).

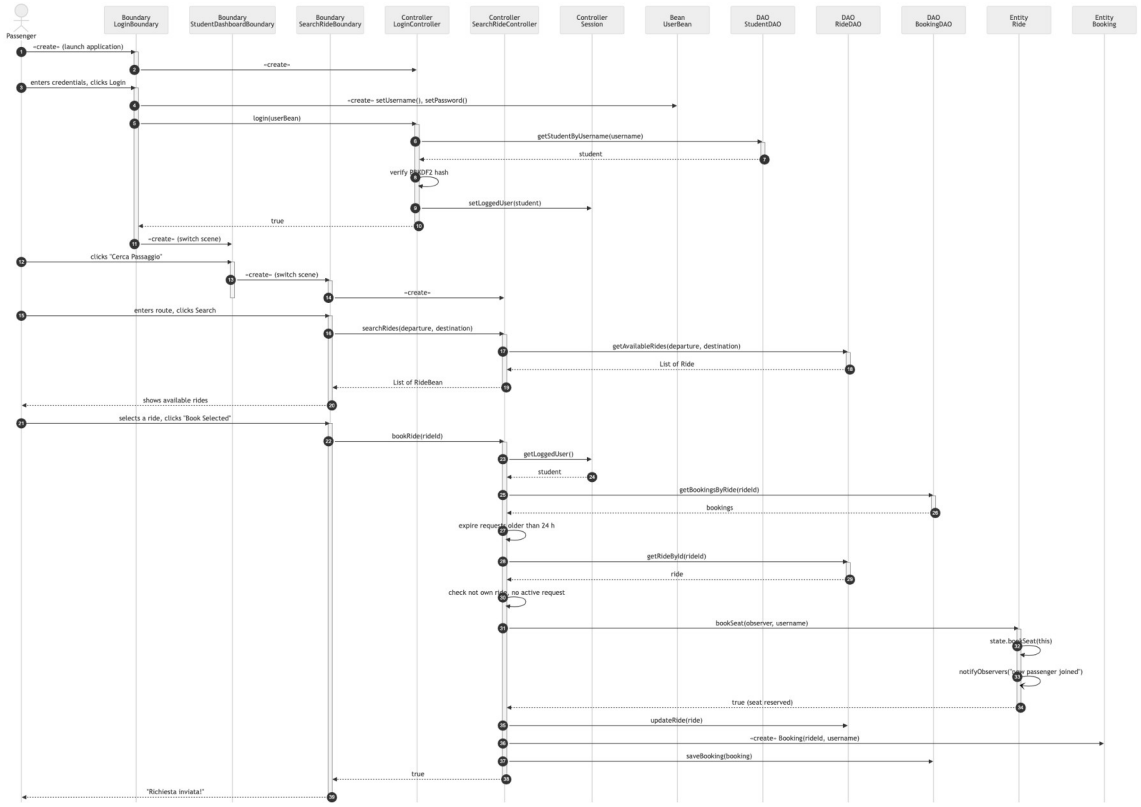


Figure 11 — Sequence Diagram of the login → search → booking request flow

3.6. State Diagram

Two state machines govern the system. The first describes the lifecycle of a Ride: while seats remain, bookSeat() and cancelSeat() keep the ride AVAILABLE; booking the last seat moves it to FULL, and freeing one brings it back to AVAILABLE. Completing the ride rejects its pending requests, while cancelling it cancels the bookings and notifies the passengers:

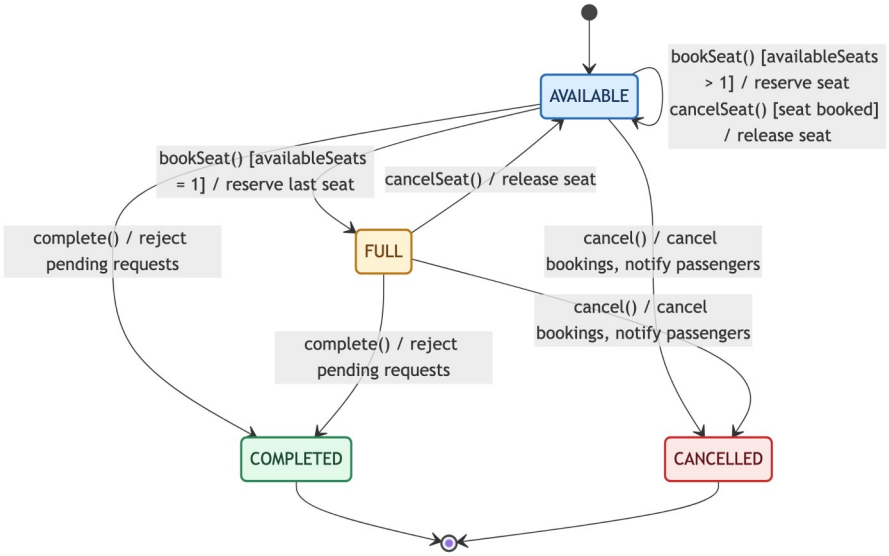


Figure 12 — State Diagram of Ride

The second describes the lifecycle of a Booking. Rejection, cancellation by the passenger and expiry release the seat reserved on the associated Ride; the after(24 h) time event models the automatic expiry of a request left unanswered, while rideCompleted() and rideCancelled() are the effects of the driver completing or cancelling the whole ride:

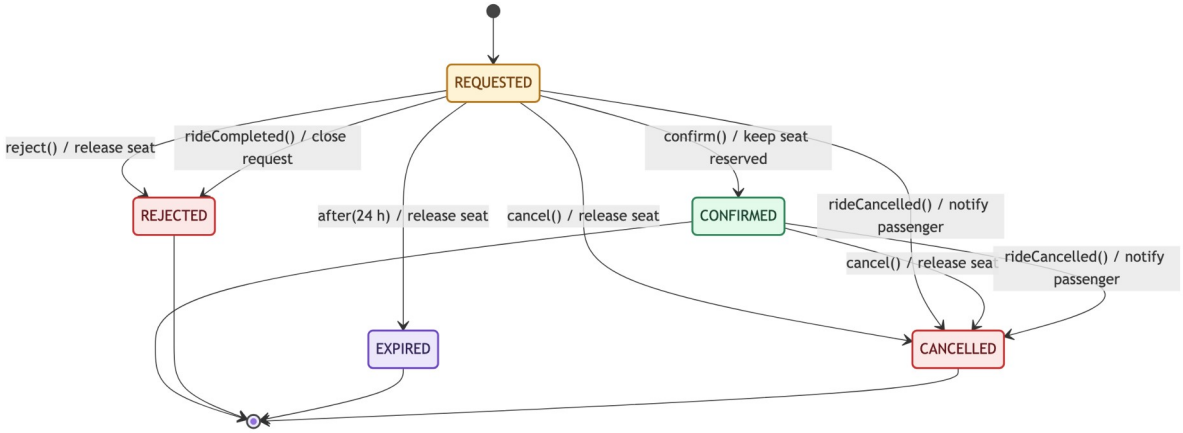


Figure 13 — State Diagram of Booking

4. TESTING

In addition to the classes that verify the domain layer and the applied patterns, the suite includes a dedicated test class for each DAO family (In-Memory, File System and SQL), so as to also cover the persistence layer and not just the business logic.

Test Class	Test Cases
RideStateTest	initial AVAILABLE state; transition to FULL on the last seat; booking rejected when the ride is full; AVAILABLE state restored on cancellation
EqualSplitStrategyTest	equal cost split with multiple passengers; no passengers (cost entirely borne by the driver); 50/50 split with one passenger

BookingWorkflowTest	new request in the REQUESTED state; confirmation of a pending request; rejection with seat release; inability to confirm/reject a request that has already been handled; cancellation by the passenger
RideLifecycleTest	completion from AVAILABLE and from FULL; a completed state is terminal (no new bookings, no double transition); cancellation even with passengers on board and the corresponding Observer notification; a cancelled state is terminal
JdbcRideDAOTest, JdbcStudentDAOTest, JdbcBookingDAOTest	saving and reading back through the real H2 database (upsert via MERGE INTO, reconstruction of a Ride's state and passengers, application of a Booking's transitions); every test uses unique identifiers (System.nanoTime()) to stay isolated even though the H2 file is shared across runs
SqlDAOFactoryTest	verifies that SqlDAOFactory actually returns the Jdbc*DAO implementations and not a stub
PasswordHasherTest	a password verifies only against its own PBKDF2 hash; two hashes of the same password differ (random salt); the legacy format is still accepted and corrupted values are rejected; the demo user can log in
CompensationTest	with a simulated failure of the BookingDAO, a booking request does not consume the seat: the change already written to the Ride is undone
ManageRidesControllerTest	confirm/reject workflow; cancelling a ride cancels its bookings and completing it rejects pending requests; a request left unanswered for more than 24 hours expires, frees the seat and can no longer be confirmed
OfferRideControllerTest, LoginControllerTest	format checks performed by the Beans (time, date, numbers, commas, username, password, phone) separated from the business rules checked by the controllers (date not in the past, duplicate username)

The domain-layer tests directly instantiate the Entities (Ride, Booking) without going through the DAOs or the Session, so they remain fully deterministic and independent of the machine they run on; the Jdbc*DAO tests, by contrast, use the real embedded H2 database, proving that the SQL layer is not just declared but actually functional. The project also includes the JaCoCo plugin to generate the code coverage report, automatically read by the SonarCloud analysis (see Deliverable Links at the end of the document).

DELIVERABLE LINKS

As requested by the Final Project Detailed Instructions, the deliverables for sections 5 (Code) and 6 (Video) are not reproduced in full in this PDF, but referenced via the links below.

Source code (also includes the JUnit tests)
https://github.com/AlessioCarta/ISPW

Static code analysis (SonarCloud — 0 bugs, 0 vulnerabilities, 0 code smells)
https://sonarcloud.io/project/overview?id=AlessioCarta_ISPW

Demo video (1–2 minutes, with audio)
https://github.com/AlessioCarta/Video-UniRide